

Full Stack Take Home Assignment

Inventory reservation and order management

Build a small application where customers reserve limited stock and simulated payment events complete their orders. We will assess backend correctness, API security, frontend usability, tests, and your ability to explain engineering decisions.

Time budget: 8-12 working hours. Stop at 12 hours and document unfinished work. Use React, NestJS, TypeScript, and PostgreSQL. Choose your ORM, UI library, and project structure. A modular monolith is sufficient.

Required workflow

- Use seeded customer and operations accounts. Customers can browse products, create orders, view only their own orders, and cancel their pending orders. Operations users can list and inspect all orders.
- Each order has one product and a positive integer quantity. Calculate amounts on the server in integer minor units, snapshot the product name and unit price, and reject insufficient stock.
- Order creation reserves stock atomically. Concurrent purchases must never oversell. Idempotency-Key is required and scoped to the customer: identical retries return the same order without another reservation; changed input with the same key returns 409.
- A simulated payment webhook confirms or fails a pending order. Protect it with X-Webhook-Secret from configuration. Persist unique event IDs and process retries safely.
- Persist successful status changes with reasons and timestamps. Concurrent cancellation and payment must produce one valid terminal state and correct stock.

From	Action	To	Stock effect
PENDING	Payment succeeds	CONFIRMED	Reserved stock becomes sold
PENDING	Payment fails	FAILED	Release reservation once
PENDING	Customer cancels	CANCELLED	Release reservation once
Terminal	Later payment event	Unchanged	No stock change

CONFIRMED, FAILED, and CANCELLED are terminal. Duplicate events return DUPLICATE; a new event for a terminal order returns IGNORED. Cancelling a terminal order returns 409. Released stock becomes available again; confirmed stock remains unavailable.

Frontend and scope

Provide product browsing and order creation, customer order list and details with cancellation, and an operations list with status filtering and pagination. Include loading, empty, validation, error, and success states, keyboard access, and a responsive layout. Polling or manual refresh is enough for payment updates.

Do not implement real payments, registration, password recovery, automatic reservation expiry, or microservices. Cloud deployment is optional and can earn up to 5 bonus marks. Docker and CI receive no separate bonus points. Keep all work within the 12-hour cap.

API contract and demo data

Default base URL: `http://localhost:4000/api`. Make it configurable. Use JSON, bearer tokens, and the routes below. Product, user, and order IDs are opaque strings. Build your submitted Postman collection around your implementation of this contract; example IDs are illustrative.

Method and route	Input or access	Output
POST <code>/auth/login</code>	email, password; public	200 Login
GET <code>/products</code>	Any authenticated user	200 <code>{items: Product[]}</code>
POST <code>/orders</code>	Customer; productId, quantity; Idempotency-Key	201 Order; identical retry 200 Order
GET <code>/orders</code>	Customer; page, pageSize, status	200 Page<Order>, own orders
GET <code>/orders/:id</code>	Owning customer	200 Order
POST <code>/orders/:id/cancel</code>	Owning customer; empty body	200 Order
GET <code>/operations/orders</code>	Operations; page, pageSize, status	200 Page<Order>
GET <code>/operations/orders/:id</code>	Operations	200 Order
POST <code>/webhooks/payments</code>	X-Webhook-Secret; PaymentEvent	200 PaymentResult

List defaults: `page=1`, `pageSize=10`; `pageSize` maximum 100. Both must be positive integers. `status` is an optional uppercase order status. Use `createdAt` descending with `id` as a stable tie-breaker. Return an empty items array for pages beyond the end.

Account	Email	Password	Role
Customer A	<code>alice@example.test</code>	DemoPass123!	CUSTOMER
Customer B	<code>bob@example.test</code>	DemoPass123!	CUSTOMER
Operations	<code>ops@example.test</code>	DemoPass123!	OPERATIONS

These credentials are only for the local assessment. Hash passwords and obtain role and identity from the authenticated user. Default local webhook secret: `local-demo-webhook-secret`, read from configuration.

Product name	Unit price minor	Available quantity
Mechanical Keyboard	1500000	20
USB C Hub	800000	20
Laptop Stand	600000	20
Wireless Mouse	450000	20
Limited Edition Headphones	2500000	1

All prices use currency LKR. 100 minor units = LKR 1.00. Begin with no orders or payment events. Provide a repeatable reset and seed command. Use these product names in your demo data and submitted collection.

Request and response models

Required fields are shown below. ISO 8601 timestamps must include a timezone. Additional fields are allowed, but do not replace the contracted fields. JSON snippets illustrate the contract. Your submitted Postman response examples must come from your running implementation.

Model	Required fields
Login	accessToken: string; user: {id: string, email: string, role: CUSTOMER OPERATIONS}
Product	id: string; name: string; unitPriceMinor: nonnegative integer; currency: LKR; availableQuantity: nonnegative integer
Order	id, customerId, productId, productName: strings; quantity: positive integer; unitPriceMinor, totalMinor: nonnegative integers; currency: LKR; status; createdAt, updatedAt: timestamps; history: Transition[]
Transition	fromStatus: status null; toStatus: status; reason: ORDER_CREATED PAYMENT_SUCCEEDED PAYMENT_FAILED CUSTOMER_CANCELLED; occurredAt: timestamp
Page<Order>	items: Order[]; page: positive integer; pageSize: positive integer; total: nonnegative integer matching the active filter
PaymentEvent	eventId: nonempty string; orderId: string; type: PAYMENT_SUCCEEDED PAYMENT_FAILED
PaymentResult	eventId: string; orderId: string; outcome: APPLIED DUPLICATE IGNORED; orderStatus: current order status
Error	code: string; message: readable string

Every Order response includes its history. Begin with null -> PENDING / ORDER_CREATED. On a real transition, append one entry in the same transaction as the state and stock updates. Retries and ignored events must not append another status transition. totalMinor = quantity x unitPriceMinor.

```
Order input: {"productId":"product-keyboard","quantity":1}
Payment input: {"eventId":"event-001","orderId":"order-001",
               "type":"PAYMENT_SUCCEEDED"}
Payment output: {"eventId":"event-001","orderId":"order-001",
                "outcome":"APPLIED","orderStatus":"CONFIRMED"}
```

A webhook event ID identifies one accepted payload. Reusing it with different orderId or type returns 409 EVENT_ID_CONFLICT. A rejected invalid or unknown-order event is not accepted. Stock checks, order creation, idempotency records, event processing, and transitions must survive process restarts.

HTTP	Error code	When
400	VALIDATION_ERROR	Invalid body, missing idempotency key, invalid list query
401	UNAUTHORIZED	Missing or invalid bearer token, credentials, or webhook secret
403	FORBIDDEN	Authenticated role cannot use this route
404	NOT_FOUND	Unknown resource or another customer's order
409	INSUFFICIENT_STOCK	Quantity exceeds available stock
409	IDEMPOTENCY_CONFLICT	Same customer key reused with changed input
409	INVALID_TRANSITION	Cancellation of a terminal order
409	EVENT_ID_CONFLICT	Accepted event ID reused with changed payload

Testing and submission

Required automated checks

- Prove two simultaneous requests for the last unit cannot both succeed. Verify the final stock and number of orders against PostgreSQL; mock-only tests are insufficient for transaction safety.
- Test identical and changed-input idempotency retries, including concurrent identical requests. Verify one order, one reservation, and one initial transition.
- Test payment success, payment failure, cancellation, duplicate callbacks, later terminal-state callbacks, and cancellation racing with payment. Check stock restoration happens at most once.
- Test invalid quantities, unauthenticated calls, forbidden roles, and cross-customer order reads and cancellations. Test at least the main frontend flow and one API failure state.

Required Postman submission

Submit your own exported Postman Collection v2.1 JSON and local environment JSON, built from your implemented API. Cover every required route and the main customer and operations flows, including payment success, failure, cancellation, retries, validation errors, and authorization checks. Parameterize the base URL, demo credentials and webhook secret; capture tokens and resource IDs automatically and generate fresh idempotency keys and event IDs for each run.

Save representative successful and error responses from your running API as request examples so reviewers can inspect the output. Include assertions for HTTP status, response fields and important state changes. Clear runtime tokens and IDs and replace secrets with placeholders before export. Document environment setup, reset/seed commands and collection run order in the README. The collection must run against fresh seed data without manual token or ID copying. Sequential Postman checks do not replace automated concurrency tests.

What to submit

- A repository link or source ZIP with frontend and backend code, migrations, seed/reset command, .env.example, and a README with prerequisites and exact install, start, and test commands.
- API documentation, automated tests, and your own Postman collection plus environment JSON, with saved response outputs and assertions for your implementation. Local setup must work without paid accounts or hosted services.
- A short engineering note describing transaction boundaries, idempotency, event handling, authentication, indexes and pagination, and what you would change for production reservation expiry.
- Your actual time spent, known limitations, unfinished work, and an AI usage note naming tools, what they helped with, and how you verified the result. If you used no AI tools, say so.

Assessment and walkthrough

Area	Weight
Data correctness and concurrency	35%
API design validation and security	20%
Frontend behavior and usability	15%
Meaningful automated tests	15%
Architecture and operational reasoning	10%
Documentation and reproducibility	5%

Optional deployment bonus: up to 5 additional marks, for a maximum of 105. Earn 3 marks for a working hosted frontend and backend demonstrating customer and operations flows; 1 for PostgreSQL persistence across an application restart or redeployment; and 1 for HTTPS plus correctly configured authentication, webhook secrets, and environment variables. Provide the demo URL, API base URL, demo access details, and deployment instructions. Use demo-only data. Docker or CI alone does not earn this bonus.

Prioritize a working vertical slice, correctness, and clear reasoning within the time budget. AI assistance and standard libraries are allowed. You must understand the delivered code and explain it during a 45-minute walkthrough. We will discuss implementation choices, test evidence, tradeoffs, and any unfinished work.